

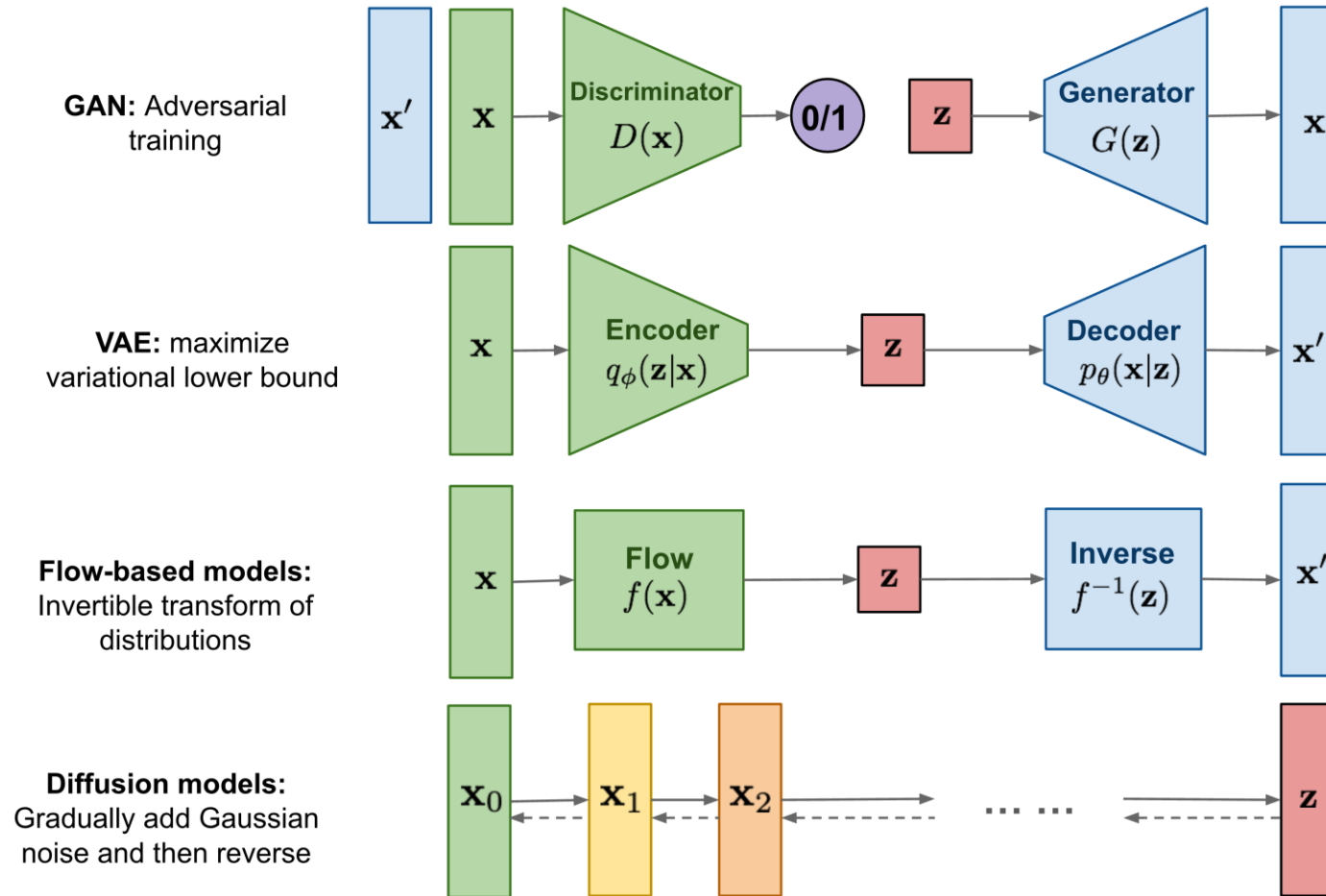
모두를 위한 딥러닝 (이미지처리편)

경북대학교 IT교육센터

허 찬

차시(모듈)	차시별 주제	학습내용
1	딥러닝을 이용한 이미지 처리	이미지 처리의 이해 다양한 이미지처리의 응용 분야에 대한 이해
2	특징 추출의 이해	이미지 데이터 전처리 기법의 이해 이미지 특징 추출의 이해 및 실습 딥러닝을 이용한 학습 및 추론 과정의 이해
3	Convolution Neural Network	합성곱 신경망의 이해
4	CNN 모델의 이해	다양한 CNN 모델의 이해 딥러닝의 여러가지 요소 이해
5	CNN을 이용한 분류 모델 실습	CNN을 이용한 이미지 분류 모델 실습
6	전이 학습 모델의 이해	전이 학습의 이해 전이학습을 이용한 이미지 처리 모델 실습
7	객체 인식 모델의 이해	다양한 객체 인식 모델의 이해 Semantic segmentation (U-Net)의 이해
8	Transformer 기반 이미지 처리 모델 1	Attention과 Transformer의 이해
9	Transformer 기반 이미지 처리 모델 2	Transformer 기반 이미지 처리 모델 (ViT)의 이해
10	이미지 생성 모델의 이해	최신 이미지 생성 모델의 이해 오토인코더의 이해 및 실습
11	최신 이미지 처리 기술의 이해	최신 컴퓨터비전 모델의 이해 비디오 처리의 이해

생성형 AI in Computer Vision



- 다양한 딥러닝 기반 생성 방법론들

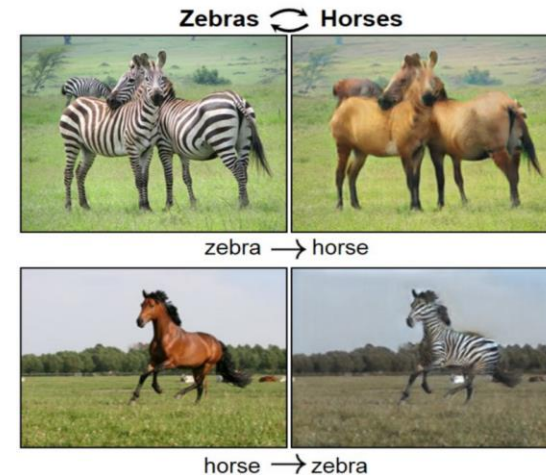
10.1 이미지를 생성하는 생성형 AI - GAN

Generative Adversarial Nets

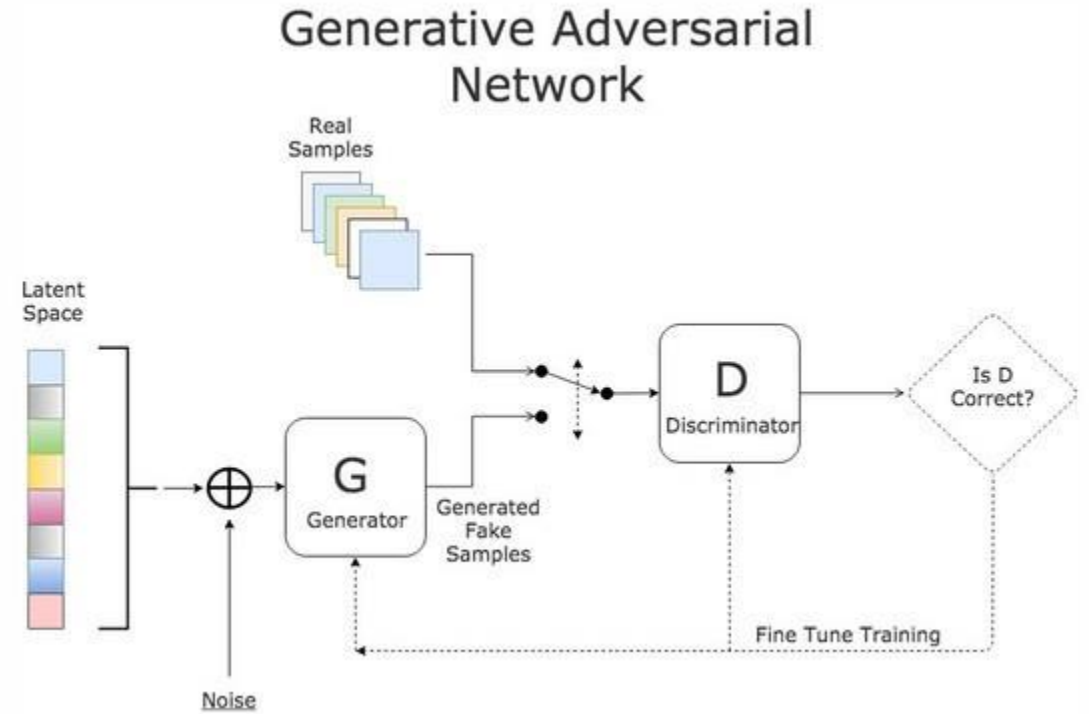
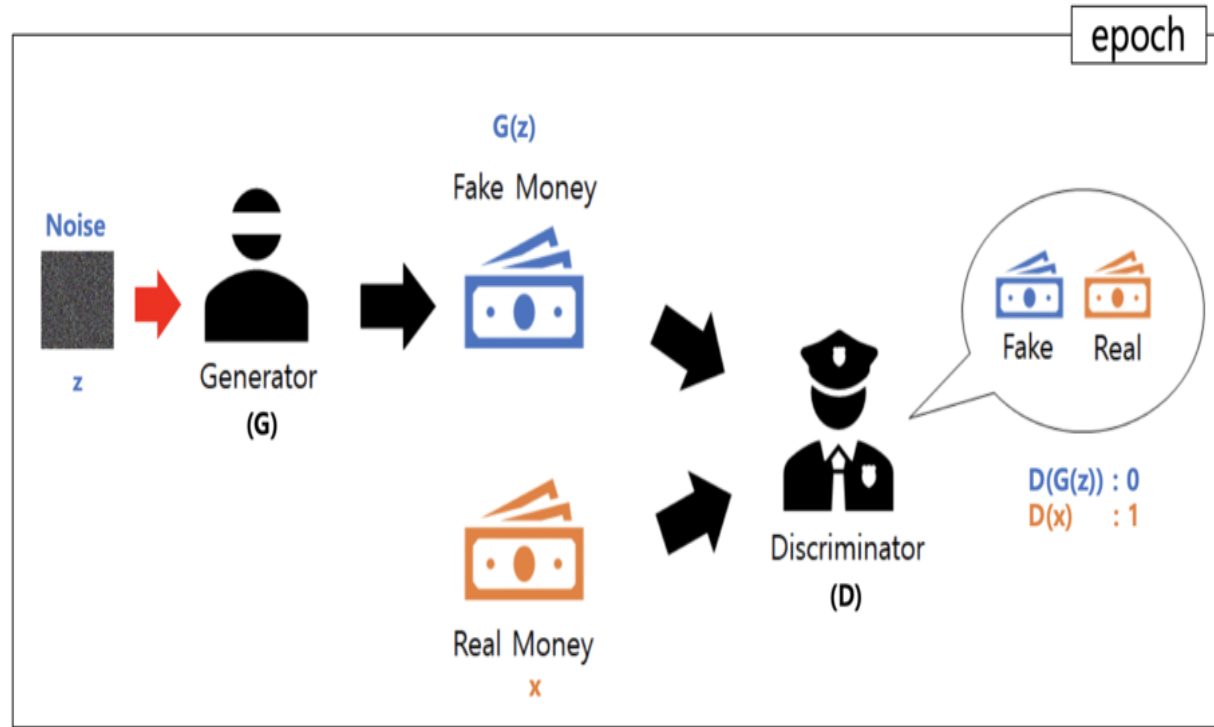
Ian J. Goodfellow, Jean Pouget-Abadie*, Mehdi Mirza, Bing Xu, David Warde-Farley,
Sherjil Ozair, Aaron Courville, Yoshua Bengio[†]
Département d'informatique et de recherche opérationnelle
Université de Montréal
Montréal, QC H3C 3J7

Abstract

We propose a new framework for estimating generative models via an adversarial process, in which we simultaneously train two models: a generative model G that captures the data distribution, and a discriminative model D that estimates the probability that a sample came from the training data rather than G . The training procedure for G is to maximize the probability of D making a mistake. This framework corresponds to a minimax two-player game. In the space of arbitrary functions G and D , a unique solution exists, with G recovering the training data distribution and D equal to $\frac{1}{2}$ everywhere. In the case where G and D are defined by multilayer perceptrons, the entire system can be trained with backpropagation. There is no need for any Markov chains or unrolled approximate inference networks during either training or generation of samples. Experiments demonstrate the potential of the framework through qualitative and quantitative evaluation of the generated samples.



10.1 이미지를 생성하는 생성형 AI - GAN



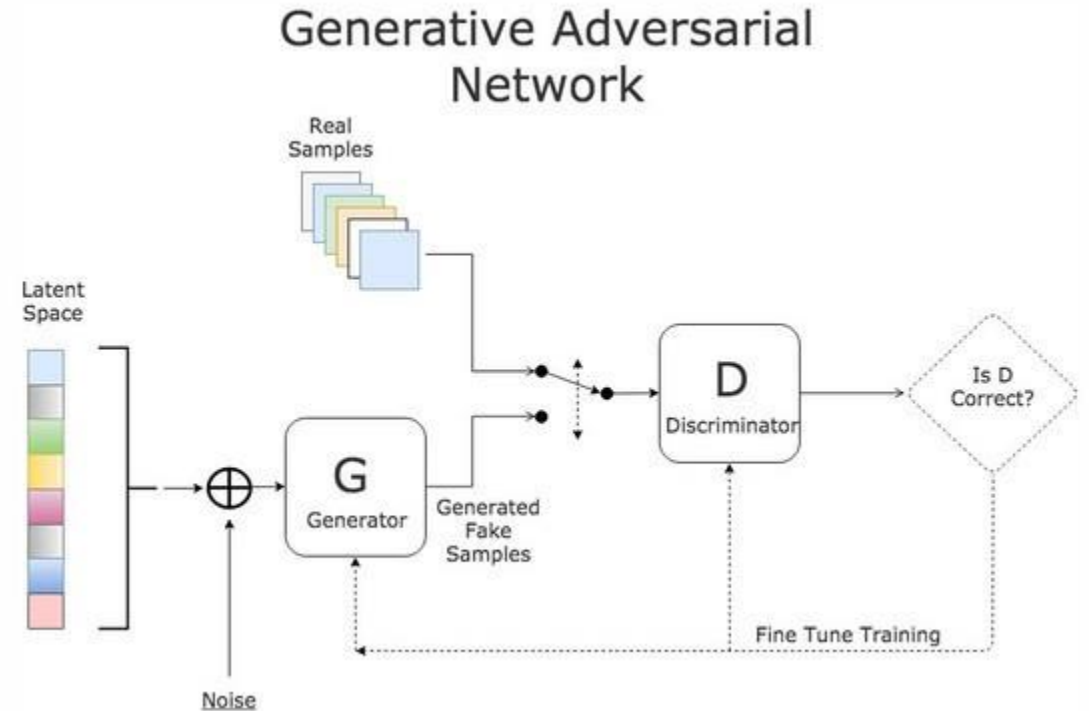
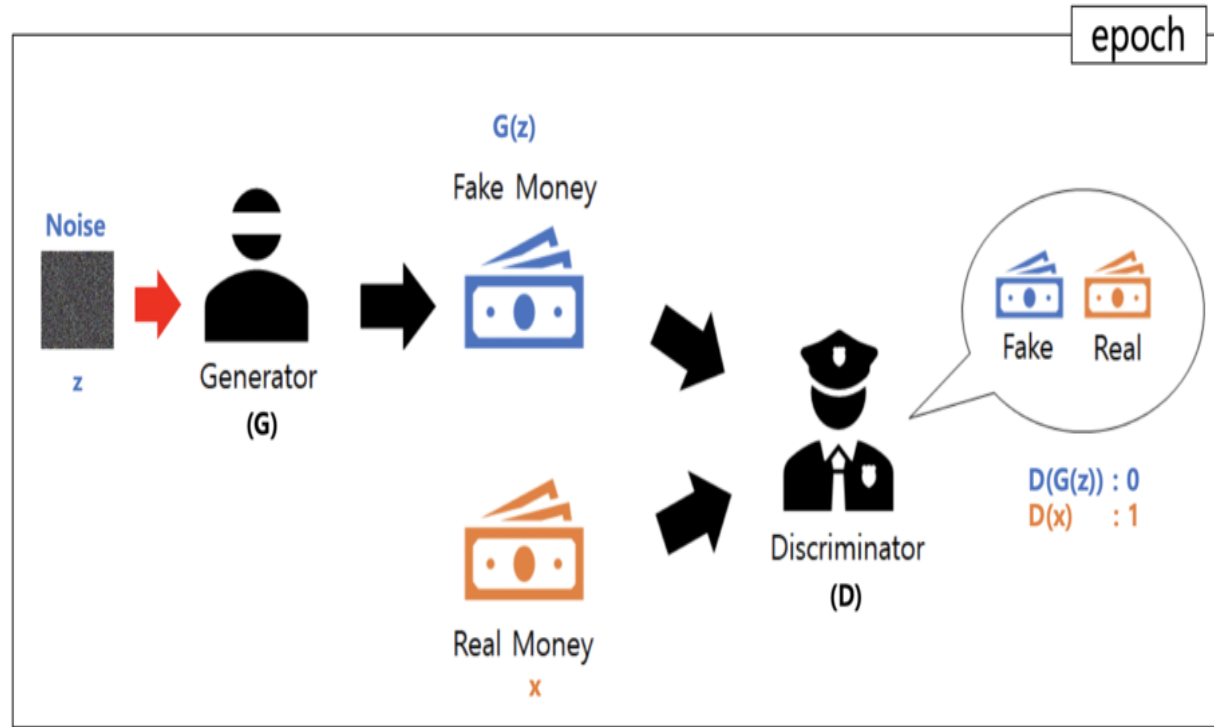
위조지폐범(Generator):

- 위조지폐를 만드는 사람. 위조지폐범은 진짜 지폐처럼 보이는 가짜 지폐를 만드는 것이 목적.

경찰(Discriminator):

- 위조지폐를 잡아내는 사람. 경찰은 지폐가 진짜인지 가짜인지 구별하는 것이 목적.

10.1 이미지를 생성하는 생성형 AI - GAN



<경쟁과 학습 과정>

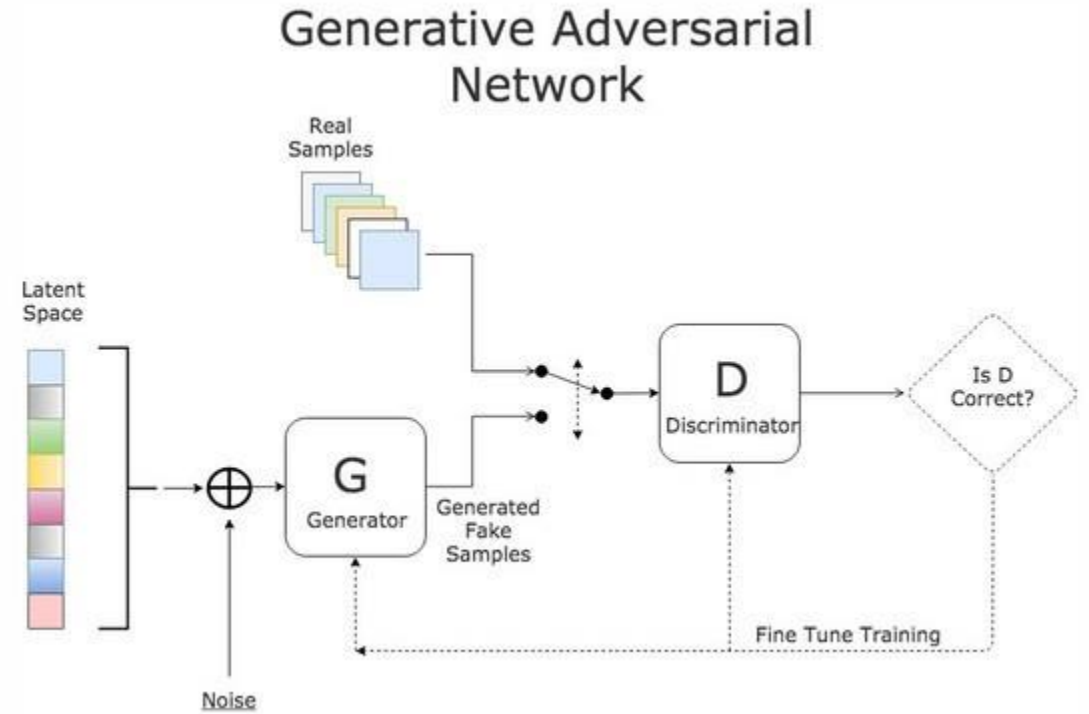
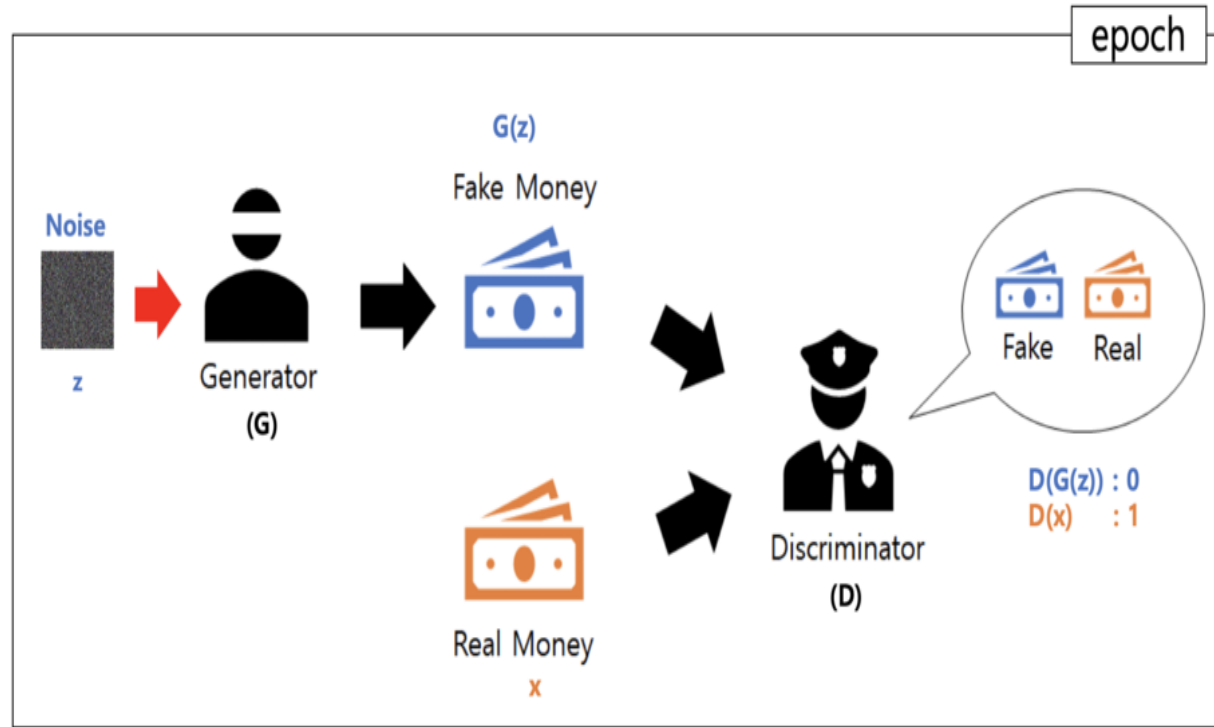
1. 위조지폐 만들기:

- 위조지폐범(생성자)은 처음에는 굉장히 서툰 위조지폐를 생성. 이 지폐는 진짜 지폐와는 많이 다름.

2. 경찰의 검증:

- 경찰(판별자)은 위조지폐범이 만든 가짜 지폐(생성 데이터)와 진짜 지폐(정답 데이터)를 비교하여 어느 것이 진짜인지 맞추려고 함. 이 과정에서 경찰은 진짜 지폐와 가짜 지폐를 구별하는 법을 배우게 됨.

10.1 이미지를 생성하는 생성형 AI - GAN



<경쟁과 학습 과정>

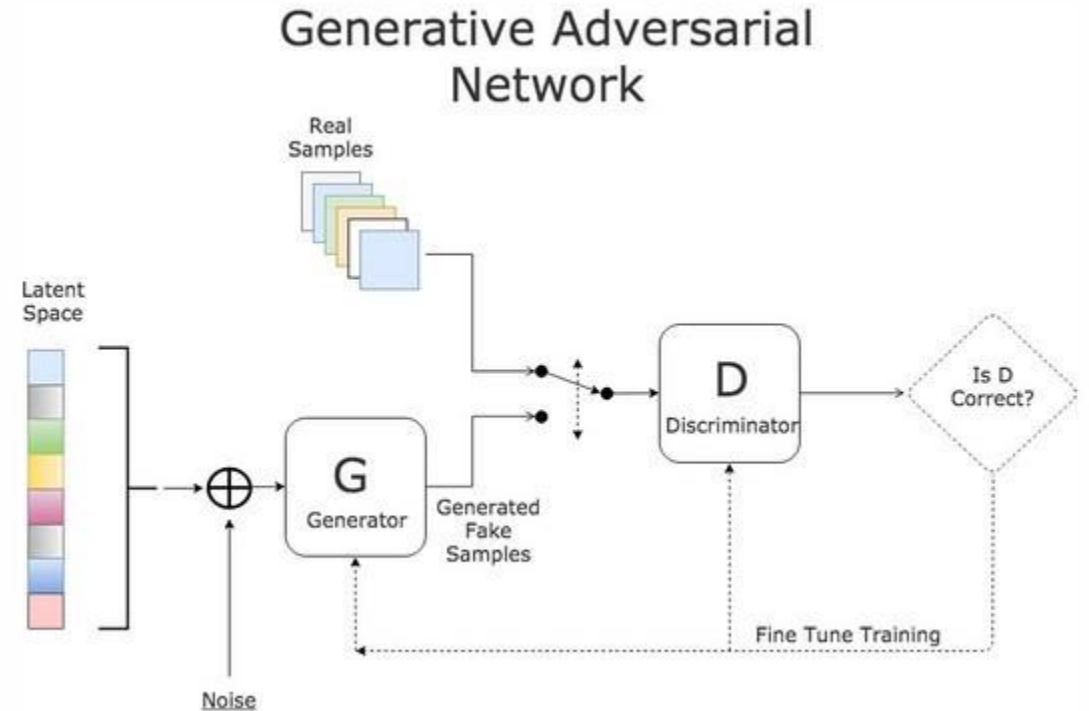
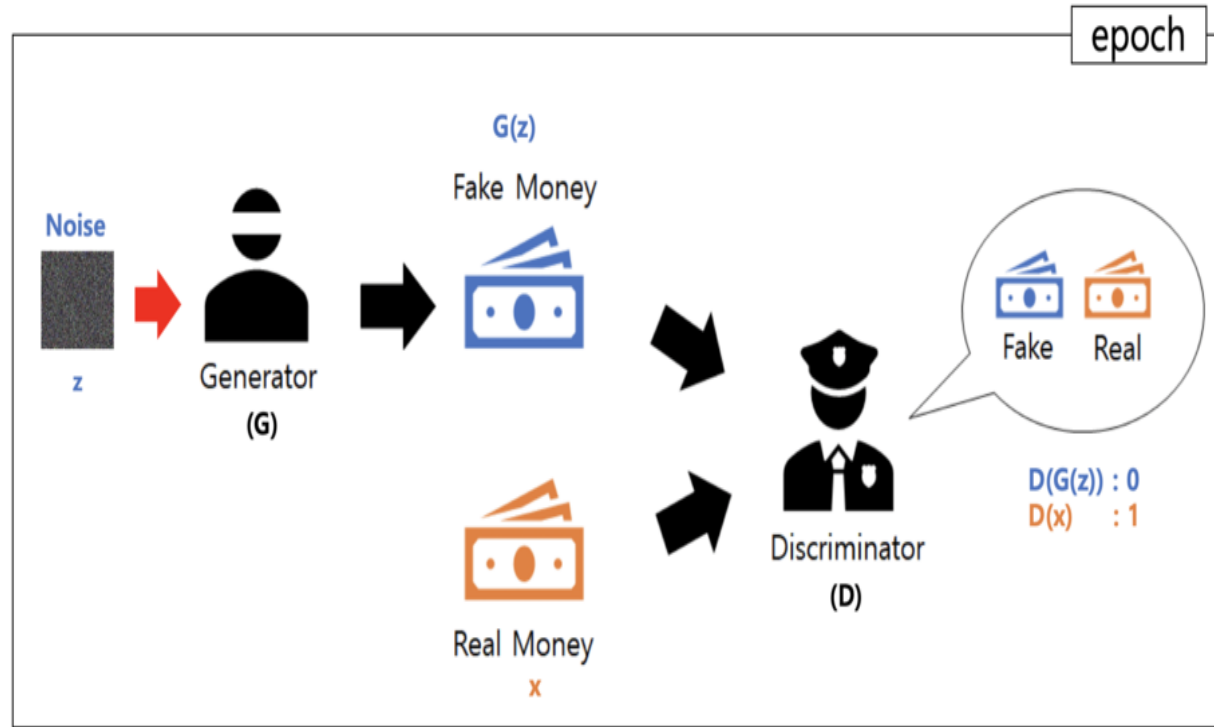
3. 경찰은 위조지폐범에게 설명 제공:

“이 지폐는 이렇게 생겼으니 가짜야” 와 같은 피드백.

4. 위조지폐 개선:

위조지폐범은 경찰의 피드백을 바탕으로 가짜 지폐를 수정하고, 점점 더 진짜 지폐처럼 보이게 만들.

10.1 이미지를 생성하는 생성형 AI - GAN



이 과정이 반복되면서, 위조지폐범은 점점 더 진짜 같은 지폐를 만들게 되고, 경찰도 점점 더 예리하게 진짜와 가짜를 구별할 수 있게 됨.

5. 최종 목표:

위조지폐범(생성자): 경찰이 구별할 수 없을 정도로 진짜 같은 가짜 지폐를 만드는 것 (50:50).

경찰(판별자): 아무리 정교하게 만든 가짜 지폐라도 진짜 지폐와 구별할 수 있는 것 (50:50).

10.1 이미지를 생성하는 생성형 AI - GAN

GAN (Generative Adversarial Network)

- **경쟁적 학습 방식:** 가짜 이미지를 생성하는 생성자(Generator)와 진짜 이미지를 판별하는 판별자(Discriminator)
- **장점:** 사실적인 이미지 생성에 강점, 다양한 이미지 스타일 표현 가능
- **단점:** 학습 과정 불안정, 모드 붕괴(Mode Collapse) 발생 가능성
- **대표적인 GAN 모델:** StyleGAN, BigGAN, ProGAN 등

10.2 이미지를 생성하는 생성형 AI - VAE

Auto-Encoding Variational Bayes

Diederik P. Kingma
Machine Learning Group
Universiteit van Amsterdam
dpkingma@gmail.com

Max Welling
Machine Learning Group
Universiteit van Amsterdam
welling.max@gmail.com

Abstract

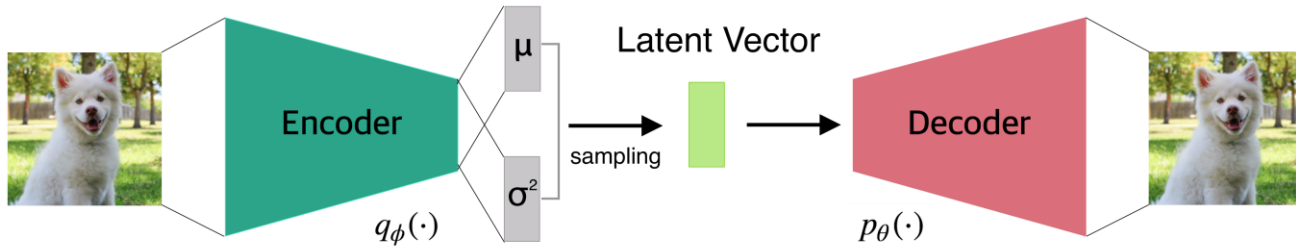
How can we perform efficient inference and learning in directed probabilistic models, in the presence of continuous latent variables with intractable posterior distributions, and large datasets? We introduce a stochastic variational inference and learning algorithm that scales to large datasets and, under some mild differentiability conditions, even works in the intractable case. Our contributions are two-fold. First, we show that a reparameterization of the variational lower bound yields a lower bound estimator that can be straightforwardly optimized using standard stochastic gradient methods. Second, we show that for i.i.d. datasets with continuous latent variables per datapoint, posterior inference can be made especially efficient by fitting an approximate inference model (also called a recognition model) to the intractable posterior using the proposed lower bound estimator. Theoretical advantages are reflected in experimental results.

1 Introduction

How can we perform efficient approximate inference and learning with directed probabilistic models whose continuous latent variables and/or parameters have intractable posterior distributions? The variational Bayesian (VB) approach involves the optimization of an approximation to the intractable posterior. Unfortunately, the common mean-field approach requires analytical solutions of expectations w.r.t. the approximate posterior, which are also intractable in the general case. We show how a reparameterization of the variational lower bound yields a simple differentiable unbiased estimator of the lower bound; this SGVB (Stochastic Gradient Variational Bayes) estimator can be used for efficient approximate posterior inference in almost any model with continuous latent variables and/or parameters, and is straightforward to optimize using standard stochastic gradient ascent techniques.

10.2 이미지를 생성하는 생성형 AI - VAE

Variational Autoencoder

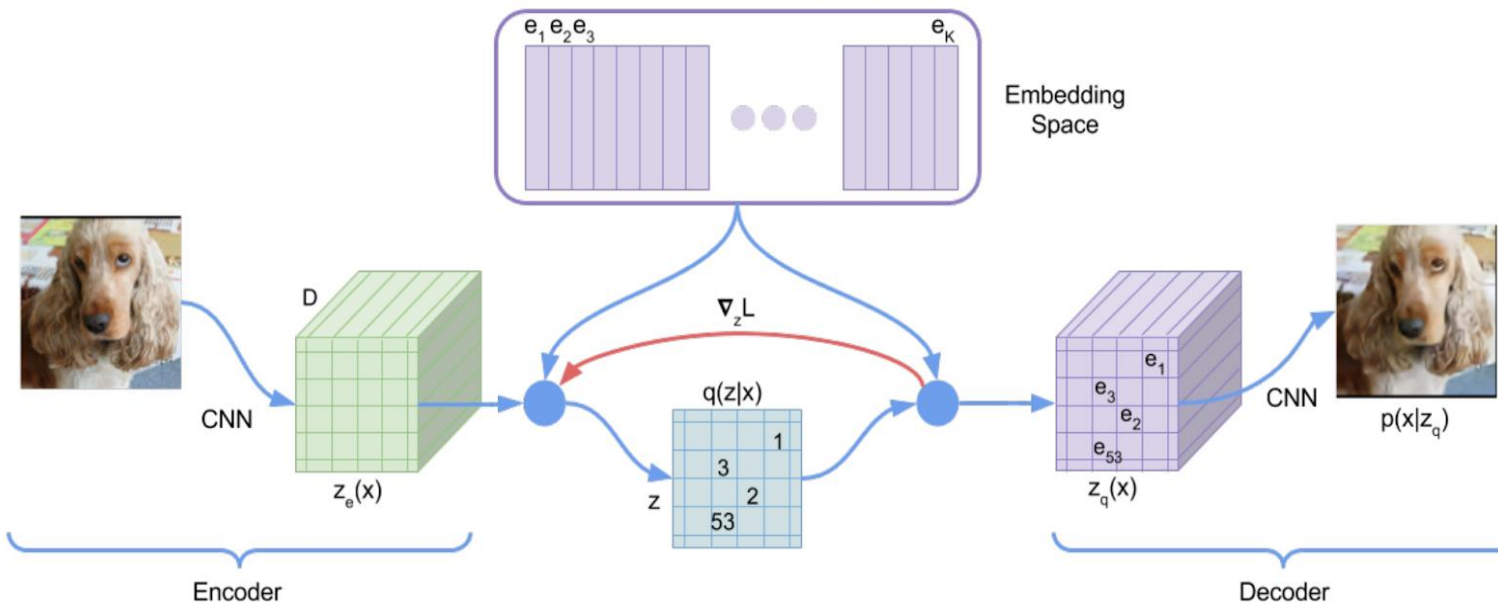


VAE:

- 인코더-디코더 구조에서 고차원의 입력 데이터를 latent vector로 압축 > 복원

VQ-VAE:

- Latent vector가 아닌 Codebook vector를 이용하여 잠재 공간을 구성. 인코딩 > 디코딩 프로세스는 같음.



10.2 이미지를 생성하는 생성형 AI - VAE

VQ-VAE (Vector Quantized Variational Autoencoder)

- **자동 인코더 기반:** 이미지를 벡터로 변환하고, 변환된 벡터를 기반으로 새로운 이미지 생성
- **장점:** 코드북 기반 이미지 생성으로 효율적, 벡터 조작으로 이미지 변형 가능
- **단점:** 저해상도 이미지 생성, 사실적인 이미지 생성 어려움
- **대표적인 VQ-VAE 모델:** VQ-VAE, VAE-GAN 등.

10.2 이미지를 생성하는 생성형 AI - Diffusion

v2 [cs.CV] 13 Apr 2022

High-Resolution Image Synthesis with Latent Diffusion Models

Robin Rombach¹ * Andreas Blattmann¹ * Dominik Lorenz¹ Patrick Esser² Björn Ommer¹

¹Ludwig Maximilian University of Munich & IWR, Heidelberg University, Germany ²Runway ML
<https://github.com/CompVis/latent-diffusion>

Abstract

By decomposing the image formation process into a sequential application of denoising autoencoders, diffusion models (DMs) achieve state-of-the-art synthesis results on image data and beyond. Additionally, their formulation allows for a guiding mechanism to control the image generation process without retraining. However, since these models typically operate directly in pixel space, optimization of powerful DMs often consumes hundreds of GPU days and inference is expensive due to sequential evaluations. To enable DM training on limited computational resources while retaining their quality and flexibility, we apply them in the latent space of powerful pretrained autoencoders. In contrast to previous work, training diffusion models on such a representation allows for the first time to reach a near-optimal point between complexity reduction and detail preservation, greatly boosting visual fidelity. By introducing cross-attention layers into the model architecture, we turn diffusion models into powerful and flexi-

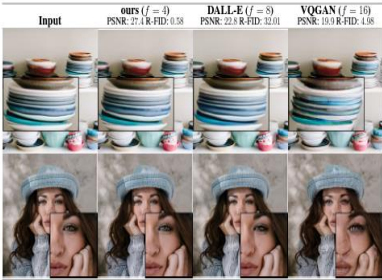


Figure 1. Boosting the upper bound on achievable quality with less aggressive downsampling. Since diffusion models offer excellent inductive biases for spatial data, we do not need the heavy spatial downsampling of related generative models in latent space, but can still greatly reduce the dimensionality of the data via suitable autoencoding models, see Sec. 3. Images are from the DIV2K [1] validation set, evaluated at 512² px. We denote the spatial downsampling factor by f . Reconstruction FIDs [29] and PSNR are calculated on ImageNet-val. [12]; see also Tab. 8.

Diffusion Models Beat GANs on Image Synthesis

Prafulla Dhariwal^{*}
OpenAI
prafulla@openai.com

Alex Nichol^{*}
OpenAI
alex@openai.com

Abstract

We show that diffusion models can achieve image sample quality superior to the current state-of-the-art generative models. We achieve this on unconditional image synthesis by finding a better architecture through a series of ablations. For conditional image synthesis, we further improve sample quality with classifier guidance: a simple, compute-efficient method for trading off diversity for fidelity using gradients from a classifier. We achieve an FID of 2.97 on ImageNet 128×128, 4.59 on ImageNet 256×256, and 7.72 on ImageNet 512×512, and we match BigGAN-deep even with as few as 25 forward passes per sample, all while maintaining better coverage of the distribution. Finally, we find that classifier guidance combines well with upsampling diffusion models, further improving FID to 3.94 on ImageNet 256×256 and 3.85 on ImageNet 512×512. We release our code at <https://github.com/openai/guided-diffusion>.

1 Introduction



High-Resolution Image Synthesis with Latent Diffusion Models (CVPR 2022)

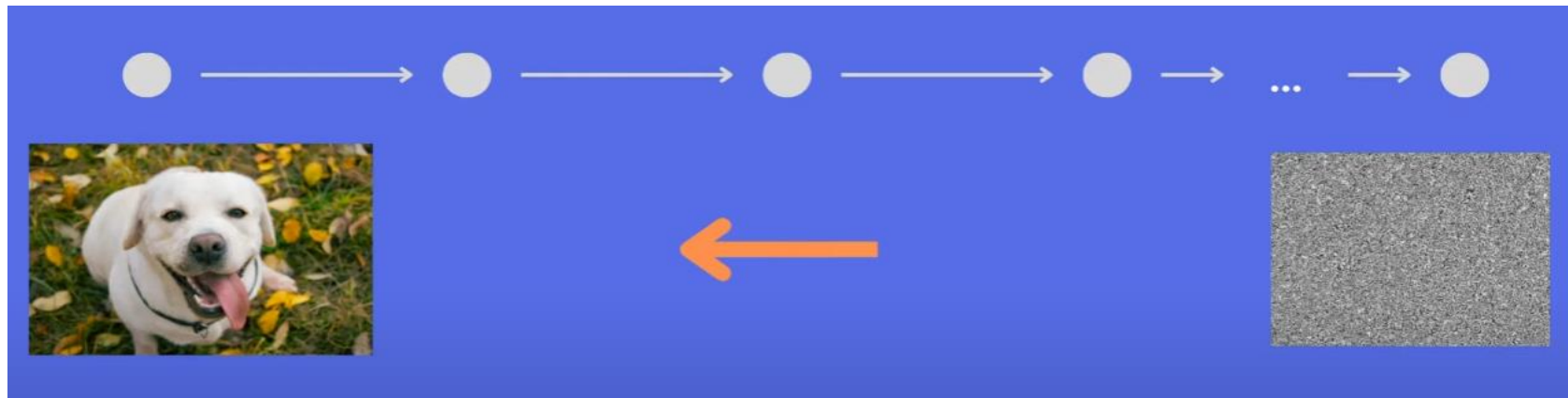
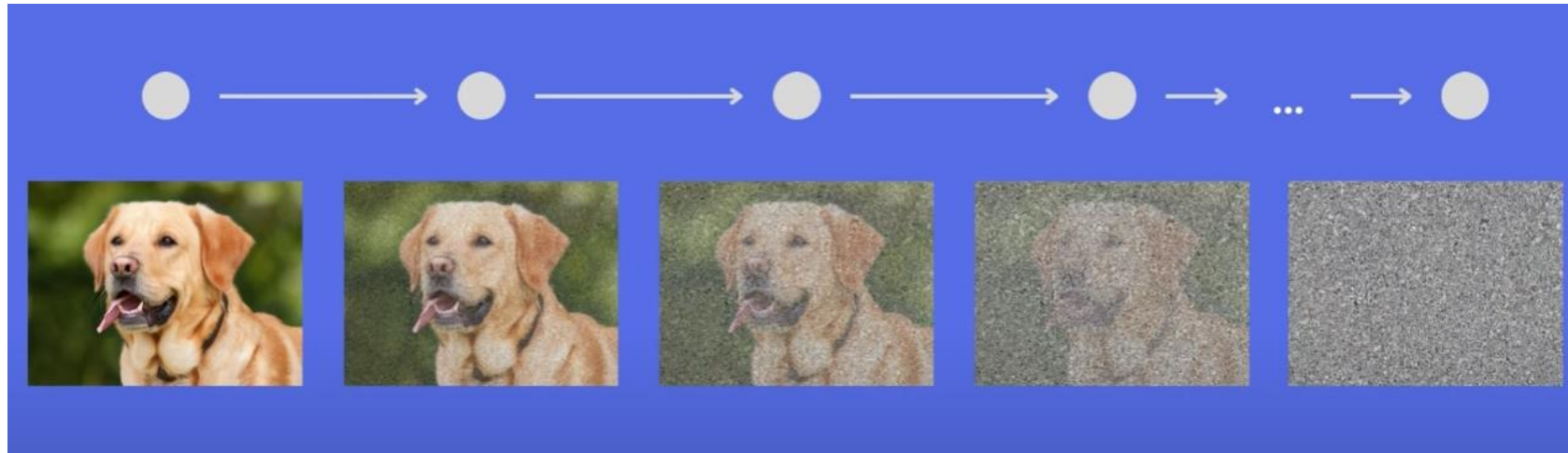
Diffusion Models Beat GANs on Image Synthesis (NIPS 2022)

10.2 이미지를 생성하는 생성형 AI - Diffusion

Diffusion Model

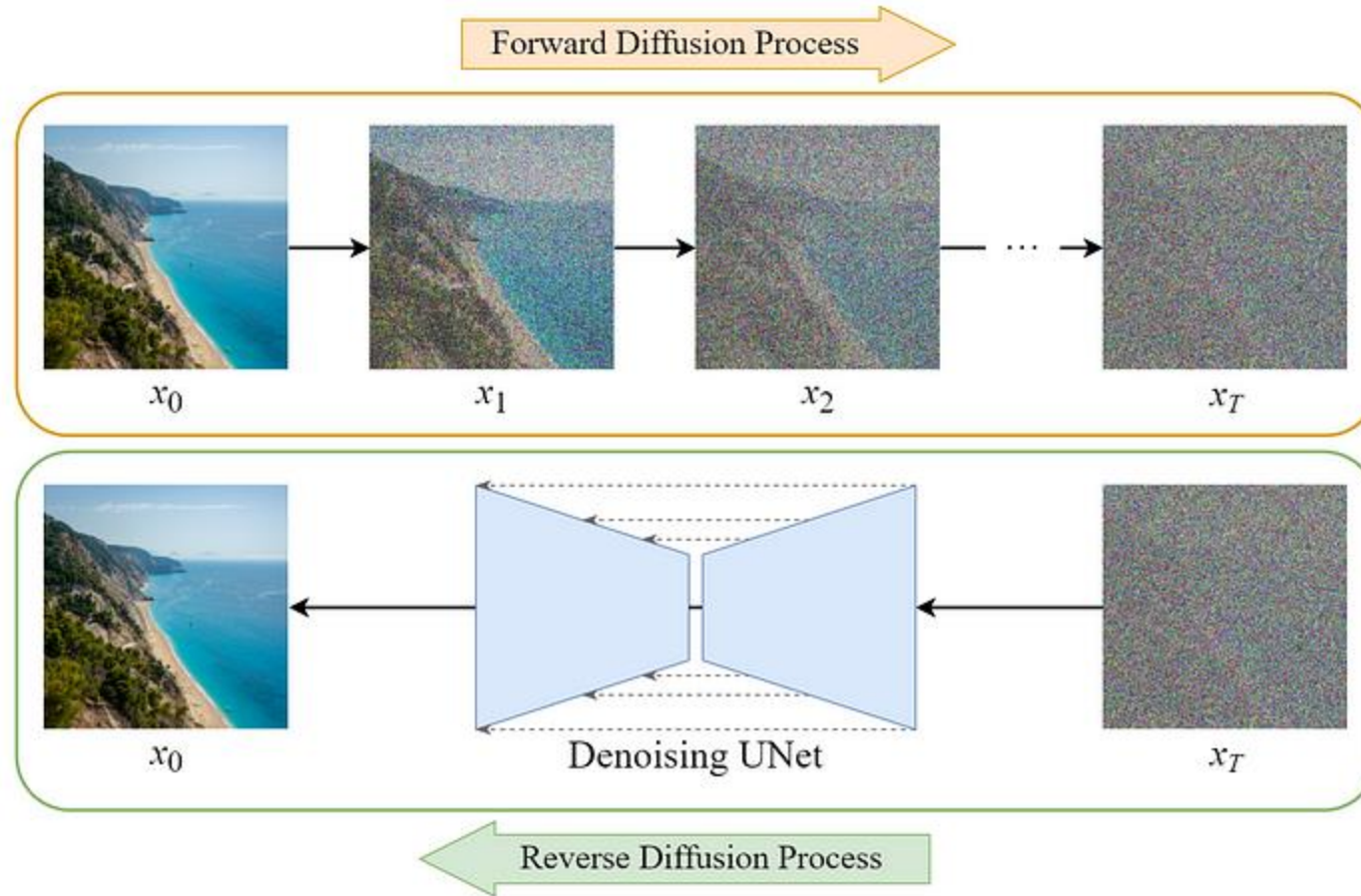
- **확산 모델:** 랜덤 노이즈에서 점차 실제 이미지로 변환하는 방식
- **장점:** 사실적인 이미지 생성에 강점, 부드러운 이미지 생성 가능
- **단점:** 학습 과정 느림, 높은 계산 비용 소요
- **대표적인 Diffusion Model:** DALL-E 2, Imagen, Stable Diffusion 등

10.2 이미지를 생성하는 생성형 AI - Diffusion



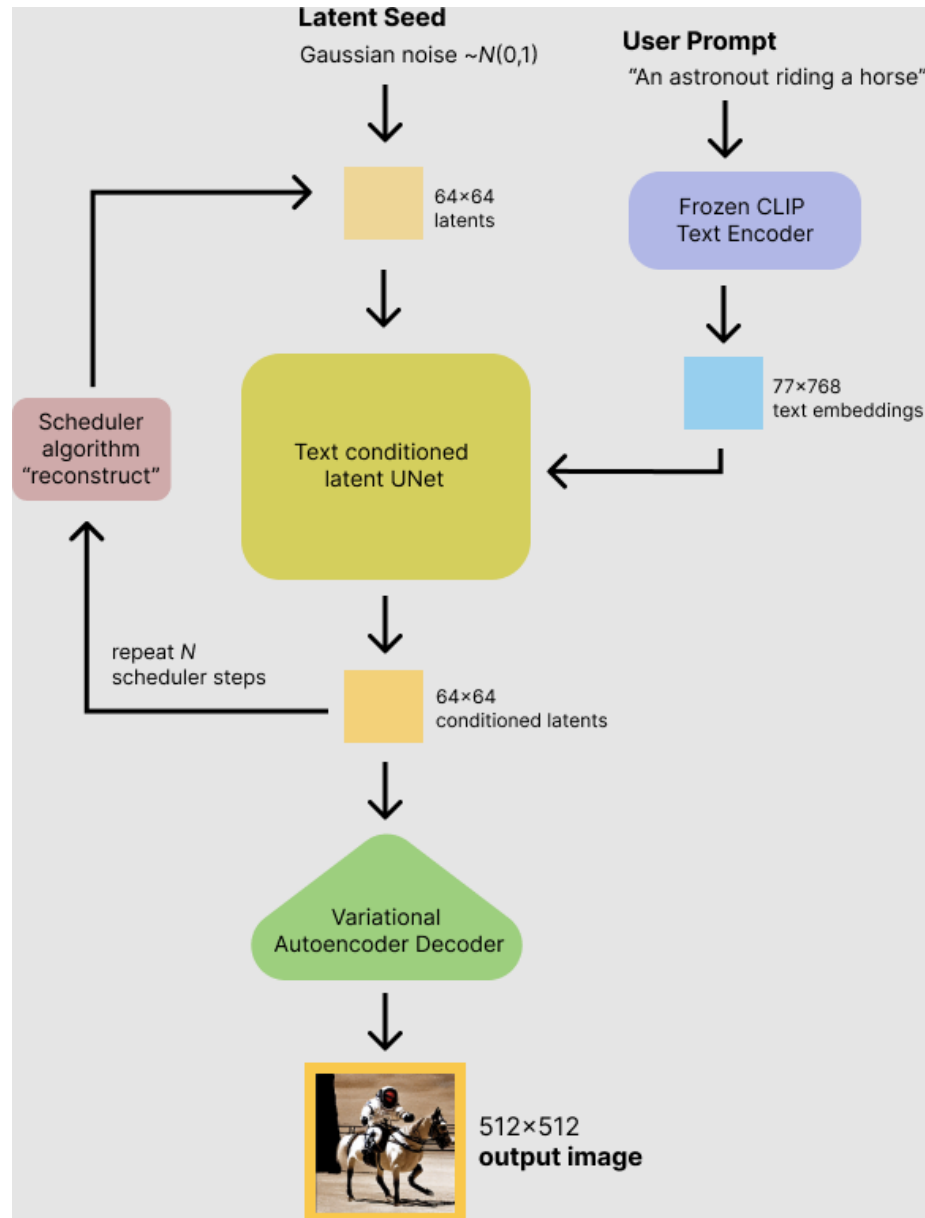
- Diffusion 기반 이미지 생성 모델

10.2 이미지를 생성하는 생성형 AI - Diffusion



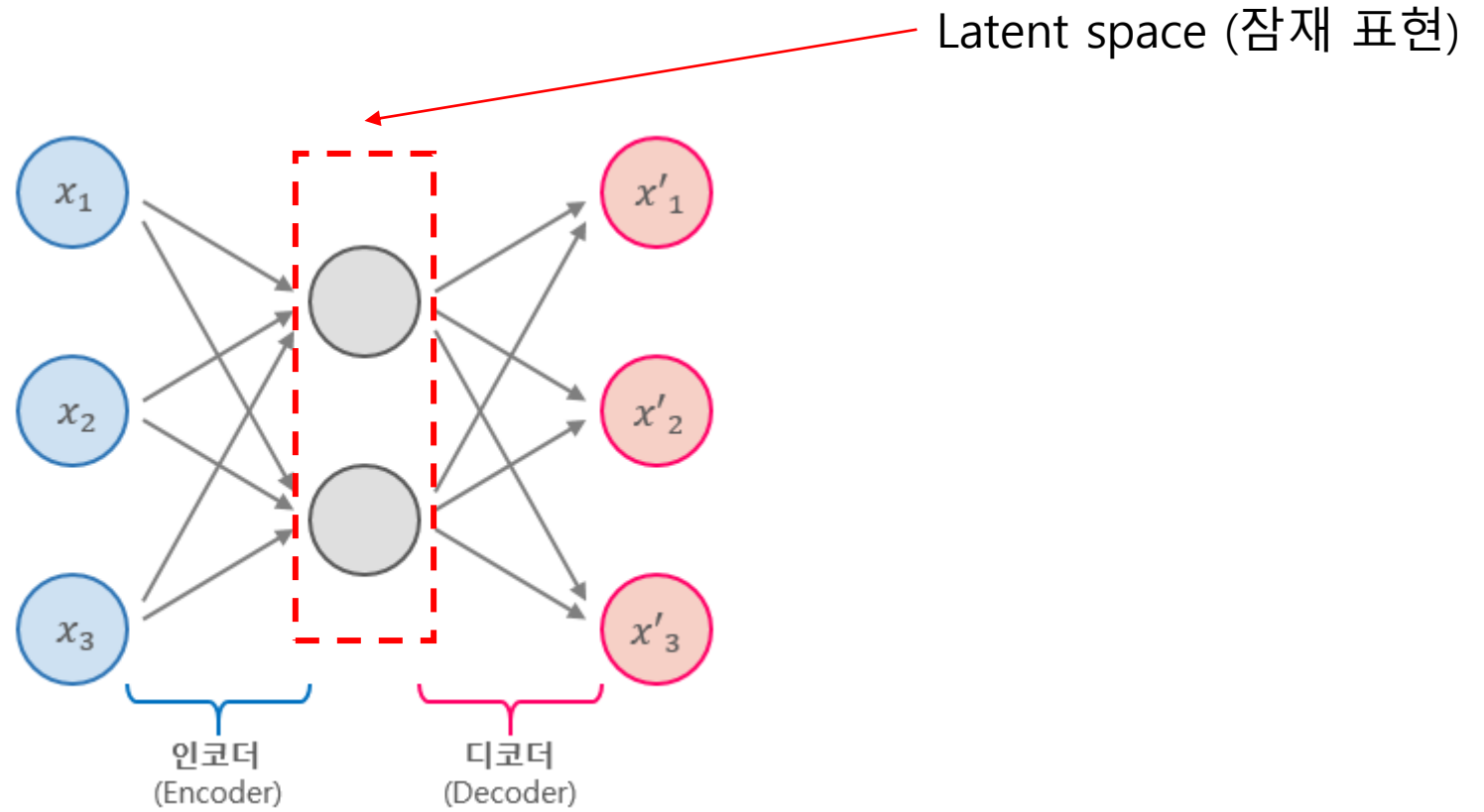
- Diffusion 기반 이미지 생성 모델

10.2 이미지를 생성하는 생성형 AI - Diffusion



- 추가적인 User prompt가 들어가야 text > image 생성 가능.

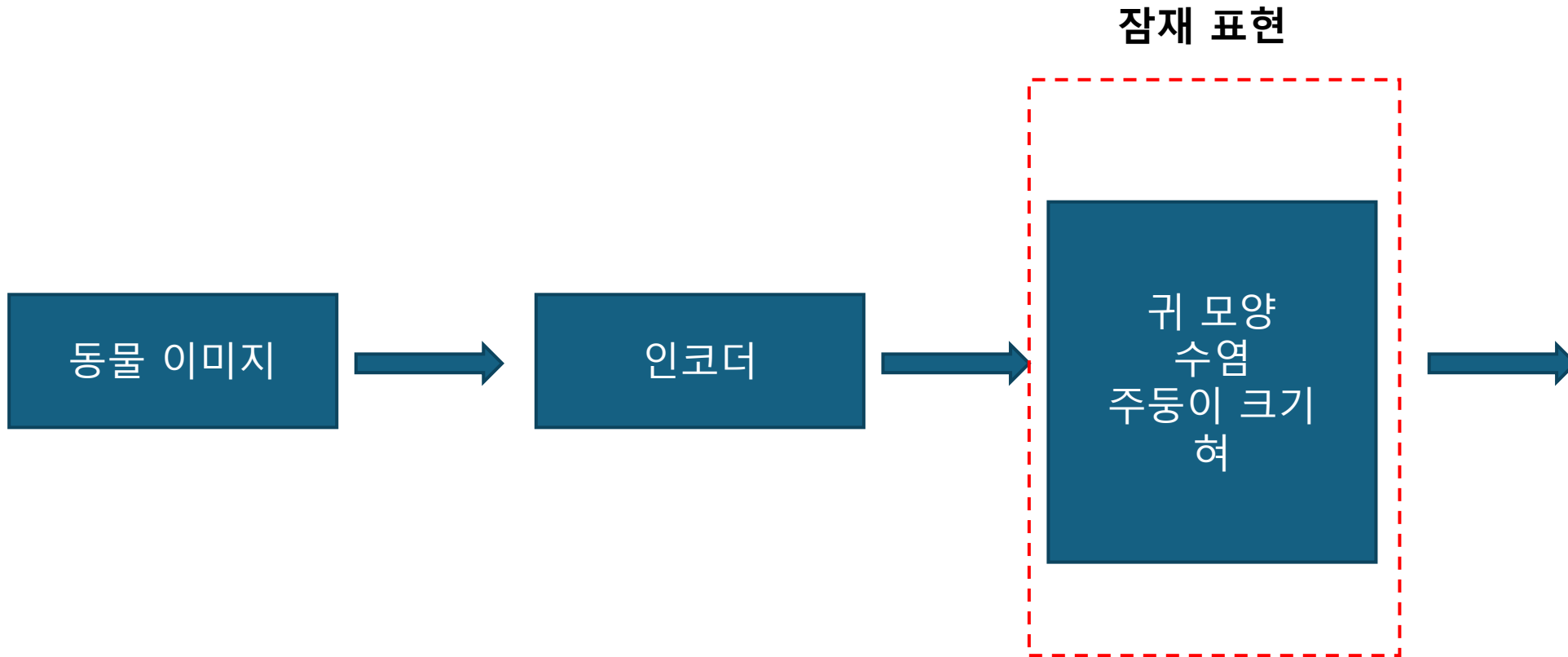
10.3 오토인코더



<AutoEncoder (오토인코더)> 모델

- 원본 입력을 압축한 잠재 표현 (latent space)를 학습

10.3 오토인코더의 이해



- 잠재 표현 : input data를 여러 개의 다른 표현의 모음으로 나타내는 방식
- Input data의 두드러진 특징들을 잡는 표현 방식

10.3 오토인코더의 이해

오토인코더의 잠재 표현의 사용 방식

- 입력 데이터의 차원을 줄여 연산량을 감소
 - >> 차원을 줄일때 최적의 값을 찾기 위해 여러 번의 실험 필요
- 데이터의 두드러진 특징 이외의 부분을 제거 (denosing)

10.3 오토인코더의 이해

오토인코더를 사용한 데이터 압축



- Left) Lossless compression (무손실 압축) / Right) Lossy compression (손실 압축)
- 잠재 벡터 표현은 손실 압축

10.3 기본 오토인코더 실습

Task 설명

- 목적: 오토인코더 모델을 이용하여 원본 데이터 복원

Input:

- 28x28 픽셀로 이루어진 이미지 데이터

Output:

- 28x28 픽셀로 이루어진 이미지 원본 데이터



MNIST dataset

10.3 기본 오토인코더 실습

1) 패키지 import

```
# 사용할 라이브러리 불러오기
import matplotlib
from keras.datasets import mnist
from keras.models import Sequential
from keras.layers import Dense
from matplotlib import pyplot as plt
import numpy as np
import random
```


10.3 기본 오토인코더 실습

2) datast load 및 전처리

```
# MNIST 데이터셋을 가져온다
training_set, testing_set = mnist.load_data()
X_train, y_train = training_set
X_test, y_test = testing_set

# 신경망에 사용할 수 있게 벡터 형태를 바꾼다
X_train_reshaped = X_train.reshape((X_train.shape[0], X_train.shape[1]*X_train.shape[2]))
X_test_reshaped = X_test.reshape((X_test.shape[0], X_test.shape[1]*X_test.shape[2]))

# 0부터 255까지의 값을 0과 1 사이 값으로 정규화한다
X_train_reshaped = X_train_reshaped/255.
X_test_reshaped = X_test_reshaped/255.
```

10.3 기본 오토인코더 실습

3) 모델 구성 및 학습

```
def create_basic_autoencoder(hidden_layer_size):
    model = Sequential()
    model.add(Dense(units=hidden_layer_size, input_shape=(784,), activation='relu'))
    model.add(Dense(units=784, activation='sigmoid'))
    return model

# 은닉 레이어 크기가 다른 오토인코더를 만든다
hiddenLayerSize_1_model = create_basic_autoencoder(hidden_layer_size=1)
hiddenLayerSize_2_model = create_basic_autoencoder(hidden_layer_size=2)
hiddenLayerSize_4_model = create_basic_autoencoder(hidden_layer_size=4)
hiddenLayerSize_8_model = create_basic_autoencoder(hidden_layer_size=8)
hiddenLayerSize_16_model = create_basic_autoencoder(hidden_layer_size=16)
hiddenLayerSize_32_model = create_basic_autoencoder(hidden_layer_size=32)

# 각 오토인코더를 훈련시킨다
hiddenLayerSize_1_model.compile(optimizer='adam', loss='mean_squared_error')
hiddenLayerSize_1_model.fit(X_train_reshaped, X_train_reshaped, epochs=10, verbose=0)

hiddenLayerSize_2_model.compile(optimizer='adam', loss='mean_squared_error')
hiddenLayerSize_2_model.fit(X_train_reshaped, X_train_reshaped, epochs=10, verbose=0)

hiddenLayerSize_4_model.compile(optimizer='adam', loss='mean_squared_error')
hiddenLayerSize_4_model.fit(X_train_reshaped, X_train_reshaped, epochs=10, verbose=0)

hiddenLayerSize_8_model.compile(optimizer='adam', loss='mean_squared_error')
hiddenLayerSize_8_model.fit(X_train_reshaped, X_train_reshaped, epochs=10, verbose=0)

hiddenLayerSize_16_model.compile(optimizer='adam', loss='mean_squared_error')
hiddenLayerSize_16_model.fit(X_train_reshaped, X_train_reshaped, epochs=10, verbose=0)

hiddenLayerSize_32_model.compile(optimizer='adam', loss='mean_squared_error')
hiddenLayerSize_32_model.fit(X_train_reshaped, X_train_reshaped, epochs=10, verbose=0)
```

10.3 기본 오토인코더 실습

4) 테스트 및 시각화

훈련시킨 모델을 사용해 테스트셋의 결과를 예측한다

```
output_1_model = hiddenLayerSize_2_model.predict(X_test_reshaped)
output_2_model = hiddenLayerSize_2_model.predict(X_test_reshaped)
output_4_model = hiddenLayerSize_4_model.predict(X_test_reshaped)
output_8_model = hiddenLayerSize_8_model.predict(X_test_reshaped)
output_16_model = hiddenLayerSize_16_model.predict(X_test_reshaped)
output_32_model = hiddenLayerSize_32_model.predict(X_test_reshaped)
```

각 모델의 결과를 그려 비교한다

```
fig, axes = plt.subplots(7, 5, figsize=(15,15))
```

```
randomly_selected_imgs = random.sample(range(output_2_model.shape[0]), 5)
```

```
outputs = [X_test, output_1_model, output_2_model, output_4_model, output_8_model, output_16_model, output_32_model]
```

서브차트를 하나씩 그린다

```
for row_num, row in enumerate(axes):
    for col_num, ax in enumerate(row):
        ax.imshow(outputs[row_num][randomly_selected_imgs[col_num]].reshape(28,28), cmap='gray')
        ax.grid(False)
        ax.set_xticks([])
        ax.set_yticks([])
```

```
plt.show()
```

10.4 오토인코더를 사용한 노이즈 제거

Denoising?



원본



노이즈



디노이징

- 노이즈(잡음) 이 있는 데이터에서 최대한 원본과 비슷하게 복원하는 작업

그림: wikipedia

10.4 오토인코더를 사용한 노이즈 제거

Task 설명

- 목적: 오토인코더 모델을 이용하여 노이즈 데이터 복원

Input:

- 28x28 픽셀로 이루어진 노이즈 이미지 데이터

Output:

- 28x28 픽셀로 이루어진 이미지 원본 데이터



MNIST dataset

10.4 오토인코더를 사용한 노이즈 제거

1) 패키지 import

#사용할 라이브러리 불러오기

```
import matplotlib
from keras.datasets import mnist
from keras.models import Sequential
from keras.layers import Dense
from matplotlib import pyplot as plt
import numpy as np
import random
```

10.4 오토인코더를 사용한 노이즈 제거

2) dataset load 및 전처리

MNIST 데이터셋을 가져온다

```
training_set, testing_set = mnist.load_data()  
X_train, y_train = training_set  
X_test, y_test = testing_set
```

신경망에 사용할 수 있게 벡터 형태를 바꾼다

```
X_train_reshaped = X_train.reshape((X_train.shape[0], X_train.shape[1]*X_train.shape[2]))  
X_test_reshaped = X_test.reshape((X_test.shape[0], X_test.shape[1]*X_test.shape[2]))
```

0부터 255까지의 값을 0과 1 사이 값으로 정규화한다

```
X_train_reshaped = X_train_reshaped/255.  
X_test_reshaped = X_test_reshaped/255.
```

MNIST 데이터셋에 노이즈를 추가한다

```
X_train_noised = X_train_reshaped + np.random.normal(0, 0.5, size=X_train_reshaped.shape)  
X_test_noised = X_test_reshaped + np.random.normal(0, 0.5, size=X_test_reshaped.shape)  
X_train_noised = np.clip(X_train_noised, a_min=0, a_max=1)  
X_test_noised = np.clip(X_test_noised, a_min=0, a_max=1)
```

10.4 오토인코더를 사용한 노이즈 제거

3) 모델 구성 및 학습

```
def create_basic_autoencoder(hidden_layer_size):
    model = Sequential()
    model.add(Dense(units=hidden_layer_size, input_shape=(784,), activation='relu'))
    model.add(Dense(units=784, activation='sigmoid'))
    return model

# 모델을 만들고 훈련시킨다
basic_denoise_autoencoder = create_basic_autoencoder(hidden_layer_size=16)
basic_denoise_autoencoder.compile(optimizer='adam', loss='mean_squared_error')
basic_denoise_autoencoder.fit(X_train_noised, X_train_reshaped, epochs=10)

output = basic_denoise_autoencoder.predict(X_test_noised)
```


10.4 오토인코더를 사용한 노이즈 제거

4) 테스트 및 시각화

훈련시킨 모델을 사용해 테스트셋의 결과를 예측한다

```
output_1_model = hiddenLayerSize_2_model.predict(X_test_reshaped)
output_2_model = hiddenLayerSize_2_model.predict(X_test_reshaped)
output_4_model = hiddenLayerSize_4_model.predict(X_test_reshaped)
output_8_model = hiddenLayerSize_8_model.predict(X_test_reshaped)
output_16_model = hiddenLayerSize_16_model.predict(X_test_reshaped)
output_32_model = hiddenLayerSize_32_model.predict(X_test_reshaped)
```

각 모델의 결과를 그려 비교한다

```
fig, axes = plt.subplots(7, 5, figsize=(15,15))
```

```
randomly_selected_imgs = random.sample(range(output_2_model.shape[0]), 5)
```

```
outputs = [X_test, output_1_model, output_2_model, output_4_model, output_8_model, output_16_model, output_32_model]
```

서브차트를 하나씩 그린다

```
for row_num, row in enumerate(axes):
    for col_num, ax in enumerate(row):
        ax.imshow(outputs[row_num][randomly_selected_imgs[col_num]].reshape(28,28), cmap='gray')
        ax.grid(False)
        ax.set_xticks([])
        ax.set_yticks([])
```

```
plt.show()
```

10.5 CNN 기반 오토인코더 실습

Task 설명

- 목적: CNN 기반 오토인코더 모델을 이용하여 노이즈 데이터 복원

Input:

- 28x28 픽셀로 이루어진 노이즈 이미지 데이터

Output:

- 28x28 픽셀로 이루어진 이미지 원본 데이터



MNIST dataset

10.5 CNN 기반 오토인코더 실습

1) 패키지 import

#사용할 라이브러리 불러오기

```
import matplotlib
from keras.datasets import mnist
from keras.models import Sequential
from keras.layers import Dense, Conv2D
from matplotlib import pyplot as plt
import numpy as np
import random
```

10.5 CNN 기반 오토인코더 실습

2) dataset load 및 전처리

MNIST 데이터셋을 가져온다

```
training_set, testing_set = mnist.load_data()  
X_train, y_train = training_set  
X_test, y_test = testing_set
```

신경망에 사용할 수 있게 벡터 형태를 바꾼다

```
X_train_reshaped = X_train.reshape((X_train.shape[0], X_train.shape[1]*X_train.shape[2]))  
X_test_reshaped = X_test.reshape((X_test.shape[0], X_test.shape[1]*X_test.shape[2]))
```

0부터 255까지의 값을 0과 1 사이 값으로 정규화한다

```
X_train_reshaped = X_train_reshaped/255.  
X_test_reshaped = X_test_reshaped/255.
```

MNIST 데이터셋에 노이즈를 추가한다

```
X_train_noisy = X_train_reshaped + np.random.normal(0, 0.5, size=X_train_reshaped.shape)  
X_test_noisy = X_test_reshaped + np.random.normal(0, 0.5, size=X_test_reshaped.shape)  
X_train_noisy = np.clip(X_train_noisy, a_min=0, a_max=1)  
X_test_noisy = np.clip(X_test_noisy, a_min=0, a_max=1)
```

10.5 CNN 기반 오토인코더 실습

3) 모델 구성 및 학습

```
conv_autoencoder = Sequential()
conv_autoencoder.add(Conv2D(filters=16, kernel_size=(3,3), activation='relu', padding='same', input_shape=(28,28,1)))
conv_autoencoder.add(Conv2D(filters=8, kernel_size=(3,3), activation='relu', padding='same'))
conv_autoencoder.add(Conv2D(filters=8, kernel_size=(3,3), activation='relu', padding='same'))
conv_autoencoder.add(Conv2D(filters=16, kernel_size=(3,3), activation='relu', padding='same'))
conv_autoencoder.add(Conv2D(filters=1, kernel_size=(3,3), activation='sigmoid', padding='same'))
conv_autoencoder.summary()
conv_autoencoder.compile(optimizer='adam', loss='binary_crossentropy')
conv_autoencoder.fit(X_train_noisy.reshape(60000,28,28,1), X_train_rescaled.reshape(60000,28,28,1), epochs=10)
```

10.5 CNN 기반 오토인코더 실습

```
output = conv_autoencoder.predict(X_test_noisy.reshape(10000,28,28,1))

# 결과 출력
fig, ((ax1, ax2, ax3, ax4, ax5), (ax6, ax7, ax8, ax9, ax10), (ax11,ax12,ax13,ax14,ax15)) = plt.subplots(3, 5)
randomly_selected_imgs = random.sample(range(output.shape[0]),5)

# 첫 번째 줄에는 원본 이미지를 그린다
for i, ax in enumerate([ax1,ax2,ax3,ax4,ax5]):
    ax.imshow(X_test_reshaped[randomly_selected_imgs[i]].reshape(28,28), cmap='gray')
    if i == 0:
        ax.set_ylabel("Original \n Images")
    ax.grid(False)
    ax.set_xticks([])
    ax.set_yticks([])

# 두 번째 줄에는 노이즈를 추가한 이미지를 그린다
for i, ax in enumerate([ax6,ax7,ax8,ax9,ax10]):
    ax.imshow(X_test_noisy[randomly_selected_imgs[i]].reshape(28,28), cmap='gray')
    if i == 0:
        ax.set_ylabel("Input With \n Noise Added")
    ax.grid(False)
    ax.set_xticks([])
    ax.set_yticks([])

# 세 번째 줄에는 오토인코더가 출력한 결과 이미지를 그린다
for i, ax in enumerate([ax11,ax12,ax13,ax14,ax15]):
    ax.imshow(output[randomly_selected_imgs[i]].reshape(28,28), cmap='gray')
    if i == 0:
        ax.set_ylabel("Denoised \n Output")
    ax.grid(False)
    ax.set_xticks([])
    ax.set_yticks([])

plt.show()
```

4) 테스트 및 시각화

감사합니다